Section A: Insurance Database Queries Given the following relational database schema:

- person (driver_id, name, address)
- car (license, model, year)
- accident (report_number, date, location)
- owns (driver_id, license)
- participated (driver_id, car, report_number, damage_amount)

1. Find the total number of people who owned cars that were involved in accidents in 2015.
2. Find the number of accidents in which the cars belonging to "John Smith" were involved.
3. Delete the Range Rover belonging to "John Smith".
4. Update the damage amount for the car with license number "AABB2000" in the accident with report number "AR2197" to $3000.

- **1. Find the total number of people who owned cars that were involved in accidents in 2015.**

```
SELECT COUNT(DISTINCT o.driver_id) AS total_people
FROM owns o
JOIN participated p ON o.license = p.car
JOIN accident a ON p.report_number = a.report_number
WHERE YEAR(a.date) = 2015;
```

- **2. Find the number of accidents in which the cars belonging to "John Smith" were involved.**

```
SELECT COUNT(DISTINCT p.report_number) AS accident_count
FROM person pe
JOIN owns o ON pe.driver_id = o.driver_id
JOIN participated p ON o.license = p.car
WHERE pe.name = 'John Smith';
```

- **3. Delete the Range Rover belonging to "John Smith".**

```
DELETE FROM car
```

```
WHERE license IN (
   SELECT o.license
   FROM owns o
   JOIN person pe ON o.driver_id = pe.driver_id
   JOIN car c ON o.license = c.license
   WHERE pe.name = 'John Smith' AND c.model = 'Range Rover'
);
```

- **4. Update the damage amount for the car with license number "AABB2000" in the accident with report number "AR2197" to $3000.**

```
UPDATE participated
SET damage_amount = 3000
WHERE car = 'AABB2000' AND report_number = 'AR2197';
```

Section B: Employee and Branch Database Schema: -

Employee (Eid, Ename, Eaddress, Egender)

Works (Bid, Eid)

Branch (Bid, Bname, Salary, Baddress)

1. Create all tables and insert 5 different records (use constraints as well).
2. Display the information of an employee who works at the same address as their branch.
3. Find the second highest salary of an employee.
4. Find the top 3 employees who earn the highest salary.
5. Sort all the information of employee and branch by Ename.
6. Find the name and address of employees who work in the ‘ABC’ branch.
7. Find the total number of branches.
8. Find the average salary of employees whose names start and end with 's'.
9. Find female employees who work in the ‘ABC’ branch and earn more than the average salary.
10. Find the name of the branch where no employee works.

11. Find the highest salary among male employees.

12. Count the number of employees working in the ‘ABC’ branch.

13. Find the highest salary in each branch.

14. Calculate the total salary distributed in the ‘ABC’ branch.

15. Display employees who work in either Ktm, Cht, or Pkr and have ID < 4.

16. Increase the salary of employees by 20% whose ID is between 4 and 5.

✅ **1. Create all tables and insert 5 different records (use constraints as well)**

```
-- Employee table
CREATE TABLE Employee (
    Eid INT PRIMARY KEY,
    Ename VARCHAR(50) NOT NULL,
    Eaddress VARCHAR(100),
    Egender VARCHAR(10) CHECK (Egender IN ('Male', 'Female'))
);

-- Branch table
CREATE TABLE Branch (
    Bid INT PRIMARY KEY,
    Bname VARCHAR(50) NOT NULL,
    Salary DECIMAL(10, 2) CHECK (Salary > 0),
    Baddress VARCHAR(100)
);

-- Works table (junction table)
CREATE TABLE Works (
    Bid INT,
    Eid INT,
    PRIMARY KEY (Bid, Eid),
```

```
    FOREIGN KEY (Bid) REFERENCES Branch(Bid),
    FOREIGN KEY (Eid) REFERENCES Employee(Eid)
);

-- Insert sample data
INSERT INTO Employee VALUES
(1, 'Suresh', 'Ktm', 'Male'),
(2, 'Sita', 'Cht', 'Female'),
(3, 'Samson', 'Pkr', 'Male'),
(4, 'Sneha', 'Ktm', 'Female'),
(5, 'Suraj', 'Pkr', 'Male');

INSERT INTO Branch VALUES
(101, 'ABC', 50000, 'Ktm'),
(102, 'XYZ', 45000, 'Cht'),
(103, 'LMN', 60000, 'Pkr'),
(104, 'OPQ', 55000, 'Bkt'),
(105, 'NOP', 48000, 'Dhn');

INSERT INTO Works VALUES
(101, 1),
(101, 2),
(102, 3),
(103, 4),
(102, 5);
```

✅ **2. Display the information of an employee who works at the same address as their branch.**

```
SELECT e.*
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE e.Eaddress = b.Baddress;
```

✅ **3. Find the second highest salary of an employee.**

```
SELECT MAX(Salary) AS second_highest_salary
FROM Branch
WHERE Salary < (SELECT MAX(Salary) FROM Branch);
```

✅ **4. Find the top 3 employees who earn the highest salary.**

```
SELECT e.Eid, e.Ename, b.Salary
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
ORDER BY b.Salary DESC
LIMIT 3;
```

✅ **5. Sort all the information of employee and branch by Ename.**

```
SELECT e.*, b.*
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
ORDER BY e.Ename ASC;
```

✅ **6. Find the name and address of employees who work in the 'ABC' branch.**

```
SELECT e.Ename, e.Eaddress
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE b.Bname = 'ABC';
```

✅ **7. Find the total number of branches.**

```
SELECT COUNT(*) AS total_branches FROM Branch;
```

✅ **8. Find the average salary of employees whose names start and end with 's'.**

```
SELECT AVG(b.Salary) AS avg_salary
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE e.Ename LIKE 's%' AND e.Ename LIKE '%s';
```

✅ **9. Find female employees who work in the 'ABC' branch and earn more than the average salary.**

```
SELECT e.Ename, b.Salary
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE e.Egender = 'Female'
  AND b.Bname = 'ABC'
  AND b.Salary > (SELECT AVG(Salary) FROM Branch);
```

✅ **10. Find the name of the branch where no employee works.**

```
SELECT b.Bname
FROM Branch b
```

```
LEFT JOIN Works w ON b.Bid = w.Bid
WHERE w.Eid IS NULL;
```

✅ **11. Find the highest salary among male employees.**

```
SELECT MAX(b.Salary) AS highest_male_salary
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE e.Egender = 'Male';
```

✅ **12. Count the number of employees working in the 'ABC' branch.**

```
SELECT COUNT(*) AS employee_count
FROM Works w
JOIN Branch b ON w.Bid = b.Bid
WHERE b.Bname = 'ABC';
```

✅ **13. Find the highest salary in each branch.**

```
SELECT b.Bname, MAX(b.Salary) AS highest_salary
FROM Branch b
GROUP BY b.Bname;
```

✅ **14. Calculate the total salary distributed in the 'ABC' branch.**

```
SELECT b.Bname, COUNT(w.Eid) * b.Salary AS total_salary
FROM Branch b
JOIN Works w ON b.Bid = w.Bid
WHERE b.Bname = 'ABC'
GROUP BY b.Bname;
```

✅ **15. Display employees who work in either Ktm, Cht, or Pkr and have ID < 4.**

```
SELECT e.*
FROM Employee e
JOIN Works w ON e.Eid = w.Eid
JOIN Branch b ON w.Bid = b.Bid
WHERE b.Baddress IN ('Ktm', 'Cht', 'Pkr') AND e.Eid < 4;
```

✅ **16. Increase the salary of employees by 20% whose ID is between 4 and 5.**

```
UPDATE Branch
SET Salary = Salary * 1.2
WHERE Bid IN (
   SELECT w.Bid
   FROM Works w
   WHERE w.Eid BETWEEN 4 AND 5
);
```

Section C: Salesman, Orders, Customer Database Schema:

- Salesman (Sid, Name, Scity, Commission, Gender)

- Orders (Ono, Purchase_amount, Order_date, Cid, Sid)

- Customer (Cid, Name, CCity, Grade, Gender)

1. Create all tables and insert 5 records (use constraints as well).

2. Find all salesman whose name starts with 'S', has 'A' as the fourth character.

3. Find the highest purchase amount on date 2023-07-22 for each salesman.

4. Count customers with grades above the average grade of customers in Kathmandu.

5. Display information of the salesman with the highest commission from Chitwan.

6. Find the second highest purchase amount. 7. Find the top 3 salesmen with the highest commission.

8. Sort all salesman information by commission.

9. Find salesmen whose city matches that of the customers, and show their name and commission.

10. Find total number of orders grouped by order date.

11. Find average purchase amount of all orders.

12. Find female customers from KTM with purchases above average purchase amount.

13. Find names of salesmen who sell to customers of the same gender.

14. Find the highest commission among male salesmen.

15. Count customers with grade '5'.

16. Find the total commission of salesmen whose names start with 'P'.

✅ **1. Create all tables and insert 5 records (use constraints as well)**

```
-- Salesman table
CREATE TABLE Salesman (
   Sid INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Scity VARCHAR(50),
   Commission DECIMAL(5,2) CHECK (Commission >= 0),
   Gender VARCHAR(10) CHECK (Gender IN ('Male', 'Female'))
);
```

```sql
-- Customer table
CREATE TABLE Customer (
   Cid INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   CCity VARCHAR(50),
   Grade INT CHECK (Grade BETWEEN 1 AND 10),
   Gender VARCHAR(10) CHECK (Gender IN ('Male', 'Female'))
);

-- Orders table
CREATE TABLE Orders (
   Ono INT PRIMARY KEY,
   Purchase_amount DECIMAL(10,2),
   Order_date DATE,
   Cid INT,
   Sid INT,
   FOREIGN KEY (Cid) REFERENCES Customer(Cid),
   FOREIGN KEY (Sid) REFERENCES Salesman(Sid)
);

-- Insert records
INSERT INTO Salesman VALUES
(1, 'Sagar', 'Kathmandu', 0.15, 'Male'),
(2, 'Priya', 'Chitwan', 0.25, 'Female'),
(3, 'Suman', 'Pokhara', 0.10, 'Male'),
(4, 'Sneha', 'Kathmandu', 0.30, 'Female'),
(5, 'Sahil', 'Butwal', 0.12, 'Male');
```

```
INSERT INTO Customer VALUES
(1, 'Bikash', 'Kathmandu', 5, 'Male'),
(2, 'Rita', 'Chitwan', 6, 'Female'),
(3, 'Sita', 'Pokhara', 7, 'Female'),
(4, 'Ramesh', 'Kathmandu', 4, 'Male'),
(5, 'Anita', 'Butwal', 5, 'Female');

INSERT INTO Orders VALUES
(101, 2000.00, '2023-07-22', 1, 1),
(102, 3500.00, '2023-07-22', 2, 2),
(103, 2800.00, '2023-07-21', 3, 3),
(104, 1500.00, '2023-07-22', 4, 1),
(105, 4000.00, '2023-07-23', 5, 4);
```

✅ **2. Find all salesman whose name starts with 'S' and has 'A' as the fourth character.**

```
SELECT *
FROM Salesman
WHERE Name LIKE 'S__A%';
```

✅ **3. Find the highest purchase amount on date 2023-07-22 for each salesman.**

```
SELECT Sid, MAX(Purchase_amount) AS MaxAmount
FROM Orders
WHERE Order_date = '2023-07-22'
GROUP BY Sid;
```

✅ **4. Count customers with grades above the average grade of customers in Kathmandu.**

```
SELECT COUNT(*) AS CustomerCount
FROM Customer
WHERE Grade > (
```

```
  SELECT AVG(Grade) FROM Customer WHERE CCity = 'Kathmandu'
);
```

✅ **5. Display information of the salesman with the highest commission from Chitwan.**

```
SELECT *
FROM Salesman
WHERE Scity = 'Chitwan'
ORDER BY Commission DESC
LIMIT 1;
```

✅ **6. Find the second highest purchase amount.**

```
SELECT MAX(Purchase_amount) AS SecondHighest
FROM Orders
WHERE Purchase_amount < (
  SELECT MAX(Purchase_amount) FROM Orders
);
```

✅ **7. Find the top 3 salesmen with the highest commission.**

```
SELECT *
FROM Salesman
ORDER BY Commission DESC
LIMIT 3;
```

✅ **8. Sort all salesman information by commission.**

```
SELECT *
FROM Salesman
ORDER BY Commission;
```

✅ **9. Find salesmen whose city matches that of the customers, and show their name and commission.**

```
SELECT DISTINCT s.Name, s.Commission
FROM Salesman s
JOIN Customer c ON s.Scity = c.CCity;
```

✅ **10. Find total number of orders grouped by order date.**

```
SELECT Order_date, COUNT(*) AS TotalOrders
FROM Orders
GROUP BY Order_date;
```

✅ **11. Find average purchase amount of all orders.**

```
SELECT AVG(Purchase_amount) AS AvgAmount
FROM Orders;
```

✅ **12. Find female customers from KTM with purchases above average purchase amount.**

```
SELECT DISTINCT c.*
FROM Customer c
JOIN Orders o ON c.Cid = o.Cid
WHERE c.Gender = 'Female'
  AND c.CCity = 'Kathmandu'
  AND o.Purchase_amount > (
      SELECT AVG(Purchase_amount) FROM Orders
  );
```

✅ **13. Find names of salesmen who sell to customers of the same gender.**

```
SELECT DISTINCT s.Name
FROM Salesman s
JOIN Orders o ON s.Sid = o.Sid
JOIN Customer c ON o.Cid = c.Cid
WHERE s.Gender = c.Gender;
```

✅ **14. Find the highest commission among male salesmen.**

```
SELECT MAX(Commission) AS HighestMaleCommission
FROM Salesman
WHERE Gender = 'Male';
```

✅ **15. Count customers with grade '5'.**

```
SELECT COUNT(*) AS Grade5Customers
FROM Customer
WHERE Grade = 5;
```

✅ **16. Find the total commission of salesmen whose names start with 'P'.**

```
SELECT SUM(Commission) AS TotalCommission
FROM Salesman
WHERE Name LIKE 'P%';
```

Section D: Student and Grade Database Schema:

- Student (Sid, Sname, Sgender, Saddress)
- Result (Gid, Sid)
- Grade (Gid, Marks, Grade, Stream)

1. Create all tables and insert 5 different records (use constraints as well).
2. Display information of the student with the highest marks from Chitwan.
3. Find the second highest marks.
4. Find the top 3 students with the highest marks.
5. Sort student and grade information by Sname.

6. Find names and addresses of students who received grade 'A'.

7. Find the total number of students grouped by gender.

8. Find average marks of students whose names start and end with 's'.

9. Find a female student from KTM with marks greater than the average.

10. Find students who have not been assigned any grade.

11. Find the highest marks among male students.

12. Count students who received grade 'B'.

13. Find the highest marks in each stream.

14. Calculate total marks scored in each stream of Science and Management.

15. Display students from Ktm, Cht, or Pkr with ID < 4.

16. Increase marks by 20% for male students aged 20 whose marks are between 20–30.

✅ **1. Create all tables and insert 5 different records (with constraints)**

```
-- Student Table
CREATE TABLE Student (
   Sid INT PRIMARY KEY,
   Sname VARCHAR(50) NOT NULL,
   Sgender VARCHAR(10) CHECK (Sgender IN ('Male', 'Female')),
   Saddress VARCHAR(50)
);

-- Grade Table
CREATE TABLE Grade (
   Gid INT PRIMARY KEY,
   Marks DECIMAL(5,2) CHECK (Marks >= 0 AND Marks <= 100),
   Grade CHAR(1) CHECK (Grade IN ('A', 'B', 'C', 'D', 'F')),
   Stream VARCHAR(20) CHECK (Stream IN ('Science', 'Management'))
);
```

```sql
-- Result Table (relationship)
CREATE TABLE Result (
    Gid INT,
    Sid INT,
    PRIMARY KEY (Gid, Sid),
    FOREIGN KEY (Gid) REFERENCES Grade(Gid),
    FOREIGN KEY (Sid) REFERENCES Student(Sid)
);

-- Sample Insert
INSERT INTO Student VALUES
(1, 'Suresh', 'Male', 'Ktm'),
(2, 'Sneha', 'Female', 'Cht'),
(3, 'Sita', 'Female', 'Ktm'),
(4, 'Samir', 'Male', 'Pkr'),
(5, 'Sujan', 'Male', 'Bkt');

INSERT INTO Grade VALUES
(101, 92, 'A', 'Science'),
(102, 75, 'B', 'Management'),
(103, 88, 'A', 'Science'),
(104, 67, 'C', 'Management'),
(105, 30, 'F', 'Science');

INSERT INTO Result VALUES
(101, 1),
(102, 2),
(103, 3),
(104, 4),
```

(105, 5);

✅ **2. Display information of the student with the highest marks from Chitwan.**

```
SELECT s.*
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
WHERE s.Saddress = 'Cht'
ORDER BY g.Marks DESC
LIMIT 1;
```

✅ **3. Find the second highest marks.**

```
SELECT MAX(Marks) AS SecondHighest
FROM Grade
WHERE Marks < (SELECT MAX(Marks) FROM Grade);
```

✅ **4. Find the top 3 students with the highest marks.**

```
SELECT s.Sname, g.Marks
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
ORDER BY g.Marks DESC
LIMIT 3;
```

✅ **5. Sort student and grade information by Sname.**

```
SELECT s.*, g.*
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
```

```
ORDER BY s.Sname;
```

✅ **6. Find names and addresses of students who received grade 'A'.**

```
SELECT s.Sname, s.Saddress
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
WHERE g.Grade = 'A';
```

✅ **7. Find the total number of students grouped by gender.**

```
SELECT Sgender, COUNT(*) AS TotalStudents
FROM Student
GROUP BY Sgender;
```

✅ **8. Find average marks of students whose names start and end with 's'.**

```
SELECT AVG(g.Marks) AS AvgMarks
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
WHERE s.Sname LIKE 's%' AND s.Sname LIKE '%s';
```

✅ **9. Find a female student from KTM with marks greater than the average.**

```
SELECT s.*
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
WHERE s.Sgender = 'Female'
  AND s.Saddress = 'Ktm'
  AND g.Marks > (SELECT AVG(Marks) FROM Grade);
```

✅ **10. Find students who have not been assigned any grade.**

```
SELECT *
FROM Student
WHERE Sid NOT IN (SELECT Sid FROM Result);
```

✅ **11. Find the highest marks among male students.**

```
SELECT MAX(g.Marks) AS HighestMaleMarks
FROM Student s
JOIN Result r ON s.Sid = r.Sid
JOIN Grade g ON r.Gid = g.Gid
WHERE s.Sgender = 'Male';
```

✅ **12. Count students who received grade 'B'.**

```
SELECT COUNT(*) AS GradeBStudents
FROM Result r
JOIN Grade g ON r.Gid = g.Gid
WHERE g.Grade = 'B';
```

✅ **13. Find the highest marks in each stream.**

```
SELECT Stream, MAX(Marks) AS HighestMarks
FROM Grade
GROUP BY Stream;
```

✅ **14. Calculate total marks scored in each stream of Science and Management.**

```
SELECT Stream, SUM(Marks) AS TotalMarks
FROM Grade
WHERE Stream IN ('Science', 'Management')
GROUP BY Stream;
```

✅ **15. Display students from Ktm, Cht, or Pkr with ID < 4.**

```
SELECT *
FROM Student
WHERE Saddress IN ('Ktm', 'Cht', 'Pkr') AND Sid < 4;
```

✅ **16. Increase marks by 20% for male students aged 20 whose marks are between 20–30.**

```
UPDATE Grade
SET Marks = Marks * 1.2
WHERE Gid IN (
   SELECT r.Gid
   FROM Result r
   JOIN Student s ON r.Sid = s.Sid
   JOIN Grade g ON r.Gid = g.Gid
   WHERE s.Sgender = 'Male' AND s.Age = 20 AND g.Marks BETWEEN 20 AND 30
);
```